Machine Learning IC Design Report HW2

File structure:

```
├── hw2_1_signal
│   ├── alexnet.cpp
│   ├── Alexnet.h
│   ├── Conv3_relu_conv4_relu.h
│   ├── Conv_functions.h
│   ├── Conv_relu_mp1.h
│   ├── Conv_relu_mp2.h
│   ├── Conv_relu_mp5.h
│   ├── Fc.h
│   ├── Layer_param.h
│   ├── Makefile
│   ├── result_cat_hw2_1_signal.log
│   ├── result_dog_hw2_1_signal.log
│   ├── run
│   └── Utils.h
├── hw2_2_buffer
│   ├── alexnet.cpp
│   ├── Alexnet.h
│   ├── Conv3_relu_conv4_relu.h
│   ├── Conv_functions.h
│   ├── Conv_relu_mp1.h
│   ├── Conv_relu_mp2.h
│   ├── Conv_relu_mp5.h
│   ├── Fc.h
│   ├── Layer_param.h
│   ├── Makefile
│   ├── result_cat_hw2_2_buffer.log
│   ├── result_dog_hw2_2_buffer.log
│   ├── run
│   └── Utils.h
├── hw2_3_fifo
│   ├── alexnet.cpp
│   ├── Alexnet.h
│   ├── Conv3_relu_conv4_relu.h
│   ├── Conv_functions.h
│   ├── Conv_relu_mp1.h
│   ├── Conv_relu_mp2.h
│   ├── Conv_relu_mp5.h
│   ├── Fc.h
│   ├── Layer_param.h
│   ├── Makefile
│   ├── result_cat_hw2_3_fifo.log
│   ├── result_dog_hw2_3_fifo.log
│   ├── run
│   └── Utils.h
└── other
    ├── cat.jpg
    └── dog.jpg

10 directories, 98 files
16:01 mlchip021@ee26[~/hw2]%
```

Dog result:

```
16:01 mlchip021@ee26[~/hw2/hw2_1_signal]% make dog
./run dog.txt

        SystemC 2.3.3-Accellera --- Mar  4 2025 01:46:3
        Copyright (c) 1996-2018 by all Contributors,
        ALL RIGHTS RESERVED
Top 100 classes:
==================================================
  idx |      val | possibility | class name
--------------------------------------------------
  207 |    16.59 |       38.63 | golden retriever
  175 |    15.57 |       13.86 | otterhound
  220 |    15.36 |       11.26 | Sussex spaniel
  163 |    15.00 |        7.86 | bloodhound
  219 |    14.59 |        5.22 | cocker spaniel
  168 |    14.39 |        4.28 | redbone
  160 |    14.35 |        4.07 | Afghan hound
  213 |    14.18 |        3.46 | Irish setter
  291 |    14.10 |        3.19 | lion
  211 |    13.01 |        1.07 | vizsla
  244 |    12.81 |        0.88 | Tibetan mastiff
  216 |    12.69 |        0.78 | clumber
  200 |    12.46 |        0.62 | Tibetan terrier
  159 |    12.42 |        0.59 | Rhodesian ridgeback
  152 |    12.38 |        0.57 | Japanese spaniel
  167 |    12.01 |        0.39 | English foxhound
  208 |    11.65 |        0.28 | Labrador retriever
  294 |    11.63 |        0.27 | brown bear
  165 |    11.51 |        0.24 | black-and-tan coonhoun
  262 |    11.46 |        0.23 | Brabancon griffon
  156 |    11.40 |        0.21 | Blenheim spaniel
  185 |    11.29 |        0.19 | Norfolk terrier
  260 |    11.11 |        0.16 | chow
  267 |    11.08 |        0.16 | standard poodle
```

Cat result:

```
        SystemC 2.3.3-Accellera --- Mar  4 2025 01:46:38
        Copyright (c) 1996-2018 by all Contributors,
        ALL RIGHTS RESERVED
Top 100 classes:
==================================================
  idx |      val | possibility | class name
--------------------------------------------------
  285 |    20.21 |       96.38 | Egyptian cat
  281 |    16.14 |        1.65 | tabby
  282 |    15.73 |        1.10 | tiger cat
  287 |    14.79 |        0.43 | lynx
  728 |    14.41 |        0.29 | plastic bag
  330 |    12.73 |        0.05 | wood rabbit
  331 |    12.19 |        0.03 | hare
  457 |    10.94 |        0.01 | bow tie
  335 |    10.67 |        0.01 | fox squirrel
  463 |    10.57 |        0.01 | bucket
  478 |    10.32 |        0.00 | carton
  876 |    10.29 |        0.00 | tub
  622 |    10.18 |        0.00 | lens cap
  904 |    10.01 |        0.00 | window screen
  700 |     9.56 |        0.00 | paper towel
  278 |     9.39 |        0.00 | kit fox
    8 |     9.29 |        0.00 | hen
  284 |     9.21 |        0.00 | Siamese cat
  722 |     8.80 |        0.00 | ping-pong ball
  434 |     8.77 |        0.00 | bath towel
  452 |     8.71 |        0.00 | bonnet
  289 |     8.31 |        0.00 | snow leopard
  753 |     8.30 |        0.00 | radiator
  681 |     8.11 |        0.00 | notebook
```

Implementation approach:

Divide the Alexnet into several module ex. Conv_relu_map1.h, Fc.h

Let the same function can do the same thing, to reduce the coding numbers.

The following are the functions.

Convolution

```
vec_3d convolution(const vec_3d& input, const vec_4d& weight, const vec_1d& bias,
    const int in_channel_num, const int out_channel_num, const int kernel_size, const int stride, const int padding){

    int input_height = input[0].size();
    int input_width = input[0][0].size();
    int output_height = (input_height + 2 * padding - kernel_size) / stride + 1;
    int output_width = (input_width + 2 * padding - kernel_size) / stride + 1;

    vec_3d input_padded(in_channel_num, vec_2d(input_height + 2 * padding, vec_1d(input_width + 2 * padding, 0)));
    vec_3d output(out_channel_num, vec_2d(output_height, vec_1d(output_width, 0)));

    for(int c = 0; c < in_channel_num; c++) {
        for(int h = 0; h < input_height; h++) {
            for(int w = 0; w < input_width; w++) {
                input_padded[c][h + padding][w + padding] = input[c][h][w];
            }
        }
    }

    for(int o = 0; o < out_channel_num; o++) {
        for(int h = 0; h < output_height; h++) {
            for(int w = 0; w < output_width; w++) {
                for(int c = 0; c < in_channel_num; c++) {
                    for(int i = 0; i < kernel_size; i++) {
                        for(int j = 0; j < kernel_size; j++) {
                            output[o][h][w] += input_padded[c][h * stride + i][w * stride + j] * weight[o][c][i][j];
                        }
                    }
                }
                output[o][h][w] += bias[o];
            }
        }
    }

    return output;
}
```

Max pool

```
vec_3d max_pooling_3d(const vec_3d& input, const int input_channel_num, const int kernel_size, const int stride) {
    int output_height = (input[0].size() - kernel_size) / stride + 1;
    int output_width = (input[0][0].size() - kernel_size) / stride + 1;

    vec_3d output(input_channel_num, vec_2d(output_height, vec_1d(output_width, 0)));

    for(int c = 0; c < input_channel_num; c++) {
        for(int h = 0; h < output_height; h++) {
            for(int w = 0; w < output_width; w++) {
                float max_val = input[c][h * stride][w * stride];
                for(int i = 0; i < kernel_size; i++) {
                    for(int j = 0; j < kernel_size; j++) {
                        max_val = max(max_val, input[c][h * stride + i][w * stride + j]);
                    }
                }
                output[c][h][w] = max_val;
            }
        }
    }

    return output;
}
```

Relu

```
vec_3d relu_3d(const vec_3d& input) {
    vec_3d output(input.size(), vec_2d(input[0].size(), vec_1d(input[0][0].size(), 0)));

    for(int c = 0; c < input.size(); c++) {
        for(int h = 0; h < input[0].size(); h++) {
            for(int w = 0; w < input[0][0].size(); w++) {
                output[c][h][w] = input[c][h][w] > 0 ? input[c][h][w] : 0;
            }
        }
    }

    return output;
}

vec_1d relu_1d(const vec_1d& input) {
    vec_1d output(input.size(), 0);

    for(int i = 0; i < input.size(); i++) {
        output[i] = input[i] > 0 ? input[i] : 0;
    }

    return output;
}
```

Softmax

```
vec_1d softmax(const vec_1d& input) {
    vec_1d output(input.size(), 0);
    float sum = 0.0;

    for(int i = 0; i < input.size(); i++) {
        output[i] = exp(input[i]);
        sum += output[i];
    }

    for(int i = 0; i < input.size(); i++) {
        output[i] /= sum;
    }

    return output;
}
```

Conv_relu_mp module:

```
SC_MODULE(Conv_relu_mp1){
    // Given 4 inputs
public:
    sc_port<sc_fifo_in_if<float *> > img_i;
    sc_port<sc_fifo_in_if<bool> > valid_i;
    sc_in<bool> clk;

    // Output uses fifo port fort outer connection
    sc_port<sc_fifo_out_if<bool> > valid_o;
    sc_port<sc_fifo_out_if<float *> > img_o;

    void run(){
        while(1){
            wait();
            valid_o->write(0);
            if(valid_i->read() == 1){
                string file_dir = "../data/";

                // Read in conv1 weights and bias
                vec_4d conv1_weights(CONV1_OUT_CHANNEL_NUM, vec_3d(CONV1_IN_CHANNEL_NUM, vec_2d(CONV1_KERNEL_SIZE, vec_1d(CONV1_KERNEL_SIZE))));
                vec_1d conv1_bias(CONV1_OUT_CHANNEL_NUM);
```

Fc module:

```
SC_MODULE(Fc){
    // Given 4 inputs
    public:
    sc_port<sc_fifo_in_if<float *> > img_i;
    sc_port<sc_fifo_in_if<bool> > valid_i;
    sc_in<bool> clk;

    // Output uses fifo port fort outer connection
    sc_port<sc_fifo_out_if<bool> > valid_o;
    sc_port<sc_fifo_out_if<float *> > img_o;

    string file_dir;
    string weight_dir;
    string bias_dir;

    int fc_num;
    int input_channel_num;
    int output_channel_num;

    void run(){
        while(1){
            wait();
            valid_o->write(0);

            if(valid_i->read() == 1){
```

Challenges faced:

sc_port does not support complex dimensions in C++, such as four-dimensional vectors, which need to be converted into one-dimensional float data before being passed to the next module.

```
vec_1d convert1dToTensor1d(float* input, int tensor_size) {
    vec_1d output(tensor_size);

    for (int i = 0; i < tensor_size; ++i) {
        output[i] = input[i];
    }

    return output;
}

float* convertTensor1dTo1d(const vec_1d& input, int tensor_size) {
    float* output = new float[tensor_size];

    for (int i = 0; i < tensor_size; ++i) {
        output[i] = input[i];
    }

    return output;
}
```

Observations:

Fifo needs to use SC_THREAD to get data. If SC_METHOD is used, it may not get data.

The differences among these three channels:

In SystemC, selecting the appropriate communication channel between modules is essential for ensuring correct synchronization, timing behavior, and simulation performance. The three commonly used channel types—sc_signal, sc_buffer, and sc_fifo—serve different purposes and exhibit distinct characteristics:

- **sc_signal<T>** represents a wire-like connection that holds a single value. It is typically used for control signals and synchronous communication. Writes to sc_signal take effect in the next delta cycle, and multiple writes in the same delta cycle are not allowed, making it suitable for modeling clocked registers or FSMs.
- **sc_buffer<T>** is similar to sc_signal, but allows multiple writes within the same delta cycle by overwriting the previous value. This channel is useful for asynchronous or event-driven communication where immediate updates are

required, such as triggering events or modeling combinational logic.

- **sc_fifo<T>** is a first-in-first-out queue that can store multiple values. It supports both blocking and non-blocking operations and is ideal for modeling data streaming, pipelined processing, and communication between producer-consumer modules. FIFO channels help manage backpressure and data flow control effectively.

Choosing the right channel depends on the nature of the data being transferred and the synchronization requirements. sc_signal is best for control and synchronization, sc_buffer for immediate value updates, and sc_fifo for transmitting sequences of data in a structured and flow-controlled manner.

Comment:

After this assignment, I began to understand how to transfer data to different modules through channels and the timing and methods of using different channels. I also learned that I can use the same module to do the same thing, so that I can implement functions of different dimensions through different parameters, without having to write an extra one to save time.

Reference:

Lecture Note
Hw2 handout
Lab1 handout
ChatGPT